CODE

1.

```
import heapq


graph = {
    'A': {'B':2, 'C':4},
    'B': {'A':2, 'C':3, 'D':7, 'E':2},
    'C': {'A':4, 'B':3, 'E':3},
    'D': {'B':7, 'E':2, 'G':2},
    'E': {'B':2, 'C':3, 'D':2},
    'G': {}
}


heuristic = {
    'A':7,
    'B':6,
    'C':4,
    'D':3,
    'E':2,
    'G':0
}


def astar(start, goal):


    open_list = []


    heapq.heappush(open_list,(heuristic[start],start))


    g = {node:float('inf') for node in graph}
    g[start]=0
```

```
parent={start:None}

closed=[]

while open_list:

    f,current = heapq.heappop(open_list)

    if current in closed:
        continue

    print("\nCurrent Node :",current)
    print("g =",g[current])
    print("h =",heuristic[current])
    print("f =",g[current]+heuristic[current])

    closed.append(current)

    if current==goal:
        break

    for neighbour,cost in graph[current].items():

        new_g = g[current]+cost

        if new_g<g[neighbour]:

            g[neighbour]=new_g
            parent[neighbour]=current
```

```
            heapq.heappush(
                open_list,
                (new_g+heuristic[neighbour],neighbour)
            )

        print("Open List :",open_list)
        print("Closed List :",closed)

    path=[]

    node=goal

    while node!=None:
        path.append(node)
        node=parent[node]

    path.reverse()

    print("\nOptimal Path :",path)
    print("Total Cost :",g[goal])

astar('A','G')
```

2.

```
import math

# Leaf nodes from the given game tree
tree = [
    [3, 5, 6],    # Left MIN
    [9, 1, 2]     # Right MIN
```

```
]

def minimax(node, alpha, beta, maximizing):

    if isinstance(node, int):
        return node

    if maximizing:

        value = -math.inf

        for child in node:
            value = max(value, minimax(child, alpha, beta, False))
            alpha = max(alpha, value)

            if alpha >= beta:
                print("Pruning occurs at MAX")
                break

        return value

    else:

        value = math.inf

        for child in node:
            value = min(value, minimax(child, alpha, beta, True))
            beta = min(beta, value)

            if alpha >= beta:
                print("Pruning occurs at MIN")
```

```
            break

    return value


alpha = -math.inf
beta = math.inf

result = minimax(tree, alpha, beta, True)

print("\nFinal Minimax Value =", result)
```